Gibbs Sampling for Bayesian Inference of Epidemic Parameters in the SIR Model

MATH 212 — Numerical Analysis Final Project

Sai Nikhil Chanda

April 22, 2026

Contents

1 Introduction **2**

2 Theory **3**
- 2.1 Prerequisite . . . 3
- 2.2 The SIR Model . . . 4
- 2.3 Bayesian Inference Implementation . . . 5
- 2.4 Gibbs Sampling . . . 6
- 2.5 Convergence of the Gibbs Sampler . . . 7
- 2.6 Runge-Kutta Methods . . . 7

3 Numerical Computation **7**
- 3.1 Synthetic Data Generation . . . 7
- 3.2 Prior Specification . . . 8
- 3.3 Metropolis-within-Gibbs Implementation . . . 8
- 3.4 Implementing Runge-Kutta . . . 9

4 Conclusion **9**
- 4.1 Simulation Results . . . 9
- 4.2 Limitations and Future Work . . . 11

5 Appendix **12**
- 5.1 Simulation Code . . . 12

Abstract

In computational biology and public health, a major question to ask is the infectiousness of a disease. The classical method is to utilize the Susceptible-Infectious-Recovered (SIR) model to estimate two key parameters: the infection rate β and recovery rate γ. When data is missing or not as readily available, its common to resort to Bayesian Inference methods. Determining the posterior allows an understanding of the underlying distribution of β and γ rather than a single point estimate. However, many times solving for the posterior distribution analytically is intractable, due to the high dimensional or complex integration.

In this project, we tackle this issue by developing an understanding of the Gibbs Sample, a Markov Chain Monte Carlo (MCMC) algorithm, that samples from the joint posterior by alternatively sampling from conditional distributions. We develop the mathematical theory and pre-requisite of the Gibbs Sampler and prove its convergence behavior. We then apply the algorithm to synthetic epidemic data generated from a known SIR model, recovering posterior distributions for β and γ that are consistent with the true parameter values. Numerical experiments include simulation testing and posterior histograms. Our results demonstrate that Gibbs sampling is an effective and interpretable approach for epidemic parameter inference, with natural extensions to more complex realistic models.

1 Introduction

It's important to understand a disease's ability to spread from a public health and policy standpoint. The Susceptible-Infectious-Recovered model was originally introduced by McKendrick [1], where he partitions the populations into the 3 categories. He then describes their behavior through a system of Ordinary Differential Equations (ODEs), revolving around two parameters; the infectious rate β and recovery rate γ. The ratio $R_0 = \frac{\beta}{\gamma}$ gives insight as to how infectious a disease is, and gives a comparison to other diseases for policy makers.

To determine these parameters, accessible large amounts of data is necessary to compute estimates, which is often unavailable or too expensive. To remedy this, Bayesian Inference methods will provide us a distribution of β and γ, rather than a single-point estimate. Bayesian Inference works by calculating a *posterior distribution* of the parameters conditioned on the observed data. Having access to a distribution of these parameters gives policy makers an ability to modify policy based on the variation and the intervals of the parameters.

However, it is impossible to obtain the posterior of the distribution, as it requires us to solve an intractable integral. This is where MCMC methods come in play, which allow us to generate samples to compute an approximation of the distribution. We consider a Markov Chain setup with a stationary distribution of $\pi(\beta, \gamma)$, matching the target posterior. Utilizing the Runge-Kutta 4th order ODE approximation, we can continue to sample estimated infection numbers. In regards to the specific algorithm, we will be following the Gibbs Sampling method. Originally brought up by Geman [2], this method produces samples through the conditional distribution, alternatively. This contrasts from other methods such as the Metropolis-Hastings, which has us propose a distribution and rely on an acceptance

probability to count the sample. To solve our problem, we will need to use a combination, as will be shown later in Section 3.

Goals:

1. Introduce the SIR epidemic model and formulate a Bayesian Inference problem.
2. Develop the mathematical theory of Gibbs sampling, including its Markov chain structure and convergence.
3. Understand and implement the Runge-Kutta 4th Order ODE approximation equation.
4. Implement the Gibbs sampler in Python and apply it to recover the parameters β and γ from synthetic epidemic data.
5. Analyze numerical performance through sample means and posterior histograms
6. Discuss the algorithm's strengths, limitations, and directions for future work.

2 Theory

2.1 Prerequisite

Random Variables. A random variable X is associated with a probability distribution having density or mass function $f(x)$ satisfying

$$\int_{\mathcal{X}} f(x)\,dx = 1. \tag{1}$$

Examples include the Normal distribution

$$f(x \mid \mu, \sigma^2) = (2\pi\sigma^2)^{-1/2} \exp\left\{ -\frac{(x-\mu)^2}{2\sigma^2} \right\}, \tag{2}$$

and the Binomial distribution

$$f(x \mid n, p) = \binom{n}{x} p^x (1-p)^{n-x}. \tag{3}$$

Likelihood and Bayesian Inference. Given an i.i.d. sample $x_1, \ldots, x_n$ from a distribution with parameter $\vec{\theta}$, the likelihood function is

$$L(\vec{\theta} \mid \mathbf{x}) = \prod_{i=1}^{n} f(x_i \mid \vec{\theta}). \tag{4}$$

Maximum likelihood estimation chooses $\hat{\vec{\theta}}$ such that

$$\frac{\partial}{\partial \vec{\theta}} L(\vec{\theta} \mid \mathbf{x}) = 0. \tag{5}$$

In the Bayesian framework, $\vec{\theta}$ is treated as a random variable with prior $\pi(\vec{\theta})$. Bayes' rule gives the posterior distribution

$$f(\vec{\theta} \mid \mathbf{x}) = \frac{f(\mathbf{x} \mid \vec{\theta})\, \pi(\vec{\theta})}{f(\mathbf{x})}, \tag{6}$$

where $f(\mathbf{x})$ is the marginal likelihood.

Monte Carlo Integration. To approximate expectations under the posterior, one may draw samples $\vec{\theta}^{(1)}, \ldots, \vec{\theta}^{(m)}$ and use the empirical average. However, direct independent sampling from $f(\vec{\theta} \mid \mathbf{x})$ is typically infeasible, requiring dependent samples generated via Markov chain methods.

Markov Chains. A discrete-time Markov chain (DTMC) $\{X_t\}$ satisfies the Markov property

$$P(X_t = i_t \mid X_0 = i_0, \ldots, X_{t-1} = i_{t-1}) = P(X_t = i_t \mid X_{t-1} = i_{t-1}). \tag{7}$$

Let P denote the transition matrix with entries $p_{ij} = P(X_{t+1} = j \mid X_t = i)$. If μ_0 is the initial distribution, then

$$\mu_t = \mu_0 P^t. \tag{8}$$

A stationary distribution π satisfies $\pi = \pi P$. For a finite, irreducible, aperiodic (ergodic) DTMC:

1. a unique stationary distribution π exists,
2. all components satisfy $\pi(i) > 0$,
3. $\mu_0 P^t \to \pi$ at an exponential rate for any initial μ_0.

Ergodicity ensures that Markov chain Monte Carlo (MCMC) methods produce samples whose empirical distribution converges to the target posterior distribution.

2.2 The SIR Model

Let N denote the number of individuals in our population, where we can divide them in 3 categories:

- S: Susceptible, individuals who are capable of contracting the disease
- I: Infectious, individuals who currently are infected with the ability to spread the diease
- R: Recovered, individuals who have had the infection and are recovered from the disease

Because the three sets partition our total population, we always say that $N_t = S_t + I_t + R_t$ for any time $t \geq 0$. Over time, the amount of people in each partition will change, which allows us to relate the dynamic to system of ODEs:

$$\frac{dS}{dt} = -\beta \frac{I}{N} S, \tag{9}$$

$$\frac{dI}{dt} = \beta \frac{I}{N} S - \gamma I, \tag{10}$$

$$\frac{dR}{dt} = \gamma I. \tag{11}$$

Here, $\beta > 0$ defines the infection rate, or the rate at which suspected individuals contract a disease from an infected individual. The other component γ defines the recover rate, or rate at which infectious individuals recover from a disease. The basic reproduction number, defined as

$$R_0 = \frac{\beta}{\gamma}, \tag{12}$$

tells us the average number of infections caused by one infectious person, before they are considered recovered. A value $R_0 > 1$ means that the disease spreads faster than it recovers. For this simple SIR model to work, we need to assume:

1. During the time of the disease, there is no births or deaths,
2. Once an individual is considered recovered, they are permanently immune to the disease.

2.3 Bayesian Inference Implementation

For this project, let $\vec{\theta} = (\beta, \gamma)$ be our parameter vector. Let $\mathbf{x} = (X_1, ..., X_T)$ be our observed data of N individuals at time $t \in \{1, ..., T\}$. We model the observations as

$$x_t \mid \vec{\theta} \sim \text{Poisson}\Big(\Delta I_t(\vec{\theta})\Big), \tag{13}$$

where $\Delta I_t(\vec{\theta}) = I_t(\vec{\theta}) - I_{t-1}(\vec{\theta})$ is the expected number of new infections at time t, obtained by numerically solving the SIR ODE system (9)–(11) with parameter $\vec{\theta}$. The likelihood of the data is then

$$p(\mathbf{x} \mid \vec{\theta}) = \prod_{t=1}^{T} \frac{e^{-\Delta I_t(\vec{\theta})} \, \Delta I_t(\vec{\theta})^{x_t}}{x_t!}. \tag{14}$$

We assign independent prior distributions to β and γ:

$$\beta \sim \text{Gamma}(\alpha_\beta, \, b_\beta), \tag{15}$$

$$\gamma \sim \text{Gamma}(\alpha_\gamma, \, b_\gamma). \tag{16}$$

We use Gamma distributions because the true parameters $\beta, \gamma > 0$ and are widely used in Bayesian statistics due to their nature of modeling waiting times. This works well as our

goal is to model the rate of infection and recovery. Using (6), we can write the posterior distribution as:

$$f(\beta, \gamma \mid \mathbf{x}) = \frac{f(\mathbf{x} \mid \beta, \gamma)\, \pi(\beta)\pi(\gamma)}{f(\mathbf{x})} \tag{17}$$

Note that we assume γ and β are independent, which makes this problem more tractable. Also because from probability theory;

$$f(\mathbf{x}) = \int_0^\infty \int_0^\infty f(\mathbf{x} \mid \beta, \gamma)\, \pi(\beta)\pi(\gamma) d\beta d\gamma, \tag{18}$$

we can say that $f(\beta, \gamma \mid \mathbf{x}) \propto f(\mathbf{x} \mid \beta, \gamma)\, \pi(\beta)\, \pi(\gamma)$. There is no closed form solution for the posterior because that would entail solving the ODE analytically to determine the likelihood.

2.4 Gibbs Sampling

For the sake of the project, we will only consider the two parameter case, specifically parameters β and γ. From our independence assumption, we can easily find the joint pdf,

$$\pi(\beta, \gamma) = \pi(\beta)\pi(\gamma) = \frac{b_\beta^{a_\beta}}{\Gamma(a_\beta)} \beta^{a_\beta - 1} e^{-b_\beta \beta} \cdot \frac{b_\gamma^{a_\gamma}}{\Gamma(a_\gamma)} \gamma^{a_\gamma - 1} e^{-b_\gamma \gamma}. \tag{19}$$

However, if we wanted to find the marginal distribution in order to sample our parameter values, we would need integrate this over our variables and observed data, which would be an intractable integral.

Suppose we wish to sample from a joint distribution $\pi(\theta_1, \theta_2) = \pi(\beta, \gamma)$. The key ingredient of the Gibbs sampler is the *full conditional distribution* of each parameter given all others:

$$\pi(\beta \mid \gamma, \mathbf{x}) \propto f(\mathbf{x} \mid \beta, \gamma)\, \pi(\beta),$$
$$\pi(\gamma \mid \beta, \mathbf{x}) \propto f(\mathbf{x} \mid \beta, \gamma)\, \pi(\gamma).$$

Algorithm 1 Gibbs Sampler for SIR Parameters

1: **Input:** Observed data $\mathbf{x}$, prior hyperparameters, number of iterations M
2: **Initialize:** $\beta^{(0)}$ or $\gamma^{(0)} > 0$
3: **for** $k = 1, 2, \ldots, M$ **do**
4: Sample $\beta^{(k)} \sim \pi(\beta \mid \gamma^{(k-1)}, \mathbf{x})$ ▷ Full conditional for β
5: Sample $\gamma^{(k)} \sim \pi(\gamma \mid \beta^{(k)}, \mathbf{x})$ ▷ Full conditional for γ
6: **end for**
7: **Output:** Samples $\left\{(\beta^{(k)}, \gamma^{(k)})\right\}_{k=0}^{M}$

Remark 2.1. *When the full conditionals do not have a closed form (as is the case here, because the likelihood involves solving an ODE), we replace each Gibbs step with a Metropolis-Hastings step targeting the respective full conditional. This is called a* Metropolis-within-Gibbs *sampler and retains the convergence guarantees of the Gibbs framework.*

This remark is important as it allows us to numerically find the conditional distribution. As we see in section 3.3, the log-posterior relies on our solution to our ODE system, meaning we will need to sample with a modified Gibbs, specifically with the Metropolis-Hastings sampling method.

2.5 Convergence of the Gibbs Sampler

Theorem 2.2 (Convergence of the Gibbs Sampler; see [3]). *Let $\pi(\theta_1, \theta_2)$ be a joint distribution on $\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2$ with strictly positive density. Suppose the full conditionals $\pi(\theta_1 \mid \theta_2)$ and $\pi(\theta_2 \mid \theta_1)$ are well defined. Then the Gibbs sampler defines an ergodic Markov chain with stationary distribution π, and*

$$\left\| P\Big((\theta_1^{(k)}, \theta_2^{(k)}) \in \cdot\Big) - \pi(\cdot) \right\|_{\mathrm{TV}} \to 0 \quad \text{as } k \to \infty.$$

2.6 Runge-Kutta Methods

The last piece of mechanics needed before any sort of algorithmic computation is a way to numerically solve for the ODE at each sampling iteration. A popular method that balances accuracy and computation is the Runge-Kutta method, specifically the fourth order method. The Runge-Kutta is a iterative method that solves for $y_{n+1} = (S_{n+1}, I_{n+1}, R_{n+1})$ from $y_n = (S_n, I_n, R_n)$ using a weighted combination of 4 slope equations for each day $t = \{0, 1, 2, ..., T\}$. For a system $y_{n+1} = f(t, y_n)$, RK4 updates by computing

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{20}$$

Where

$$k_1 = hf(t_n, y_n) \tag{21}$$

$$k_2 = hf(t_n + \frac{h}{2}, y_n + \frac{k_1}{2}) \tag{22}$$

$$k_3 = hf(t_n + \frac{h}{2}, y_n + \frac{k_2}{2}) \tag{23}$$

$$k_4 = hf(t_n + h, y_n + k_3). \tag{24}$$

Here, h is the time step, which we will say is 1 day. For this to work, we need to specify an initial condition y_0.

3 Numerical Computation

3.1 Synthetic Data Generation

We generate synthetic epidemic data from the SIR model with known "true" parameters $\beta^* = 0.3$ and $\gamma^* = 0.1$, giving $R_0^* = 3.0$. The population is $N = 5000$ with initial conditions $S(0) = 4990$, $I(0) = 10$, $R(0) = 0$. We numerically integrate equations (9)–(11) using

equations (22) - (24) over $T = 60$ days, and generate observed daily incidence counts by drawing from the Poisson likelihood (13):

$$x_t \sim \text{Poisson}(\Delta I_t(\beta^*, \gamma^*)), \quad t = 1, \ldots, 60. \tag{25}$$

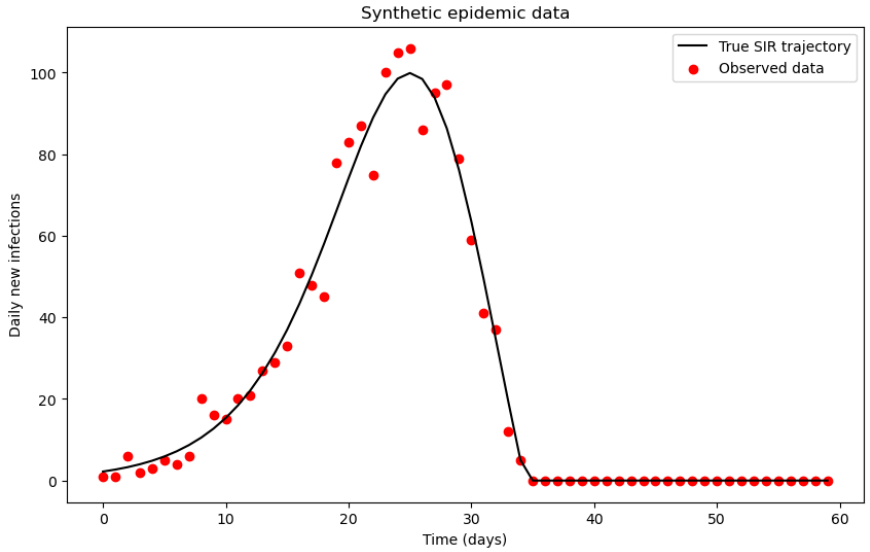


Figure 1: Synthetic epidemic data. The solid line shows the true SIR model trajectory for $\beta^* = 0.3$, $\gamma^* = 0.1$, and the points show the noisy observed daily incidence counts.

3.2 Prior Specification

We assign weakly informative Gamma priors:

$$\begin{aligned} \beta &\sim \text{Gamma}(2,\ 10), \quad (\text{mean } = 0.2,\ \text{std } \approx 0.14), \\ \gamma &\sim \text{Gamma}(2,\ 10), \quad (\text{mean } = 0.2,\ \text{std } \approx 0.14). \end{aligned}$$

These priors are centered near the true values but are broad enough to allow the data to create an inference.

3.3 Metropolis-within-Gibbs Implementation

Since the full conditionals do not have a closed form, we implement a Metropolis-within-Gibbs sampler. At each iteration, each parameter is updated via a random-walk Metropolis step with a Gaussian proposal:

$$\beta' \sim \mathcal{N}(\beta^{(k-1)},\ \sigma_\beta^2), \quad \gamma' \sim \mathcal{N}(\gamma^{(k-1)},\ \sigma_\gamma^2). \tag{26}$$

We tune the proposal standard deviations σ_β and σ_γ to achieve acceptance rates of approximately 20%.

The log-posterior is:

$$\log f(\theta \mid \mathbf{x}) = \sum_{t=1}^{T} [x_t \log \Delta I_t(\theta) - \Delta I_t(\theta)] + (\alpha_\beta - 1)\log\beta - b_\beta\beta + (\alpha_\gamma - 1)\log\gamma - b_\gamma\gamma \quad (27)$$

This comes from equation $f(\beta, \gamma \mid \mathbf{x}) \propto f(\mathbf{x} \mid \beta, \gamma)\, \pi(\beta)\, \pi(\gamma)$ and converting it to the log form.

3.4 Implementing Runge-Kutta

To implement the Runge-Kutta 4th order approximation, we refer to equations (22)-(24) for our function $f(t, S, I, R)$ with a $h = 1$ to represent daily changes. For each timestep n, we get the following:

$$k_1^S = -\beta\frac{S_n I_n}{N}, \qquad k_1^I = \beta\frac{S_n I_n}{N} - \gamma I_n, \qquad k_1^R = \gamma I_n \quad (28)$$

$$k_2^S = -\beta\frac{\left(S_n + \frac{h}{2}k_1^S\right)\left(I_n + \frac{h}{2}k_1^I\right)}{N}, \quad k_2^I = \beta\frac{\left(S_n + \frac{h}{2}k_1^S\right)\left(I_n + \frac{h}{2}k_1^I\right)}{N} - \gamma\left(I_n + \frac{h}{2}k_1^I\right), \quad k_2^R = \gamma\left(I_n + \frac{h}{2}k_1^I\right) \quad (29)$$

$$k_3^S = -\beta\frac{\left(S_n + \frac{h}{2}k_2^S\right)\left(I_n + \frac{h}{2}k_2^I\right)}{N}, \quad k_3^I = \beta\frac{\left(S_n + \frac{h}{2}k_2^S\right)\left(I_n + \frac{h}{2}k_2^I\right)}{N} - \gamma\left(I_n + \frac{h}{2}k_2^I\right), \quad k_3^R = \gamma\left(I_n + \frac{h}{2}k_2^I\right) \quad (30)$$

$$k_4^S = -\beta\frac{\left(S_n + h\,k_3^S\right)\left(I_n + h\,k_3^I\right)}{N}, \quad k_4^I = \beta\frac{\left(S_n + h\,k_3^S\right)\left(I_n + h\,k_3^I\right)}{N} - \gamma\left(I_n + h\,k_3^I\right), \quad k_4^R = \gamma\left(I_n + h\,k_3^I\right) \quad (31)$$

4 Conclusion

4.1 Simulation Results

After running the algorithm in 5.1, which utilizes equations from Section 3, we can analyze the results from our simulation. From the Metropolis-within-Gibbs sampling, the following histograms were developed, showing a distribution centered around the "true parameters" of our simulated data.

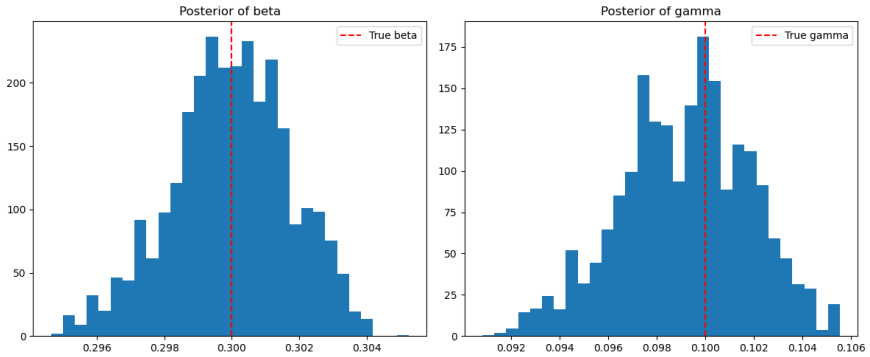


Figure 2: Histogram of parameters β and γ from Algorithm 5.1. The dashed line shows the true parameter value and the bars show frequencies of observed parameter values from the sampling process.

From the histograms, we can see there is a good centering for both parameters, with γ having a tighter and more accurate distribution. Calculating the mean and variance of both parameters gives:

Table 1: Posterior summary statistics for β and γ.

Parameter	Sample Mean $\hat{\mu}$	Sample Variance $\hat{\sigma}^2$	Acceptance Rate
β	0.3001	0.000003	15%
γ	0.0995	0.000006	19%

From the table, we can see the Metropolis-Gibbs sampling converged nicely to our "true parameters" that were used to create our synthetic data. With 5000 overall samples of each and the acceptance rates above, the ergodic theorem confirms the Gibbs sampling will provide us a marginal distribution of each parameter. We can also plot each SIR curve from our Gibbs Sampling, shown below.

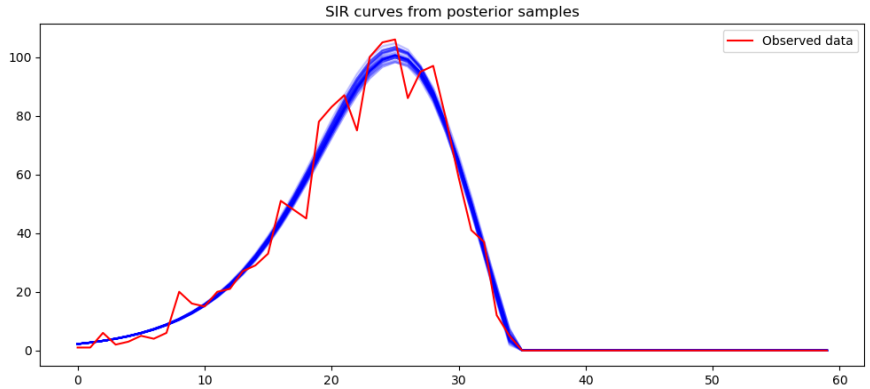


Figure 3: SIR plots for each (β_k, γ_k) with $k \in \{2001, 2002, ..., 7000\}$ with the observed data shown with the red lineplot.

4.2 Limitations and Future Work

The SIR Model is an effective and simplistic way to better understand the severity of a given disease when given observed data. However, this model was used for the purpose of demonstrating mathematical theory and is quite rudimentary. There are many extensions to this model that include vaccination procedures, temporary immunity, and other conditions to relax assumptions. More advanced stochastic processes are studied in [8], where they implement Markov jump processes and prevalence probabilities. However how simple, the SIR model is wildly accepted and many diseases have their own reproduction number and a minimum vaccination percentage. In this paper, we discussed the preliminary mathematical foundation, covered Gibbs Sampling and its application to Inference, the Runge-Kutta 4th order ODE approximation, and implemented it all in a simulation with synthetic data, deriving accurate results.

References

[1] W. O. Kermack and A. G. McKendrick, "A contribution to the mathematical theory of epidemics," *Proceedings of the Royal Society of London A*, vol. 115, no. 772, pp. 700–721, 1927.

[2] S. Geman and D. Geman, "Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-6, no. 6, pp. 721–741, 1984. DOI: 10.1109/TPAMI.1984.4767596

[3] G. Casella and E. I. George, "Explaining the Gibbs sampler," *The American Statistician*, vol. 46, no. 3, pp. 167–174, 1992. DOI: 10.1080/00031305.1992.10475878

[4] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953.

[5] P. D. O'Neill and G. O. Roberts, "Bayesian inference for partially observed stochastic epidemics," *Journal of the Royal Statistical Society: Series A*, vol. 162, no. 1, pp. 121–129, 1999.

[6] B. Nguyen-Van-Yen, P. Del Moral, and B. Cazelles, "Stochastic epidemic models inference and diagnosis with Poisson random measure data augmentation," *Mathematical Biosciences*, vol. 335, article 108583, 2021. DOI: 10.1016/j.mbs.2021.108583

[7] K. O. Roster, "Introduction to SIR Modeling," Lecture Notes, Harvard T.H. Chan School of Public Health, 2025.

[8] J. Fintzi, X. Cui, J. Wakefield, and V. N. Minin, "Efficient Data Augmentation for Fitting Stochastic Epidemic Models to Prevalence Data," *J. Comput. Graph. Stat.*, vol. 26, no. 4, pp. 918–929, 2017. doi:10.1080/10618600.2017.1328365.

[9] R. L. Burden, J. D. Faires, and A. M. Burden, *Numerical Analysis*, 10th ed. Cengage Learning, 2016, ch. 5, sec. 5.4.

5 Appendix

5.1 Simulation Code

```
   #SIR Model System of ODEs
def sir_model(S, I, R, beta, gamma, N):
    #dS/dt
    dS = -beta * S * I / N

    #dI/dt
    dI =  beta * S * I / N - gamma * I

    #dR/dt
    dR =  gamma * I
    return dS, dI, dR

#Implememntation of Rugne-Kutta 4th order method to solve the SIR
   model ODEs
def simulate_sir_rk4(beta, gamma, N=5000, S0=4990, I0=10, T=60, h
   =1.0):
    steps = int(T / h)

    #Initial conditions
    S, I, R = S0, I0, N - S0 - I0

    #Storing the trajectories
    S_list = [S]
    I_list = [I]
    R_list = [R]

    for _ in range(steps):
        # k1
        dS1, dI1, dR1 = sir_model(S, I, R, beta, gamma, N)

        # k2
        dS2, dI2, dR2 = sir_model(
            S + 0.5*h*dS1,
            I + 0.5*h*dI1,
            R + 0.5*h*dR1,
            beta, gamma, N
        )

        # k3
        dS3, dI3, dR3 = sir_model(
            S + 0.5*h*dS2,
            I + 0.5*h*dI2,
            R + 0.5*h*dR2,
```

```
            beta, gamma, N
        )

        # k4
        dS4, dI4, dR4 = sir_model(
            S + h*dS3,
            I + h*dI3,
            R + h*dR3,
            beta, gamma, N
        )

        # Update
        S += (h/6)*(dS1 + 2*dS2 + 2*dS3 + dS4)
        I += (h/6)*(dI1 + 2*dI2 + 2*dI3 + dI4)
        R += (h/6)*(dR1 + 2*dR2 + 2*dR3 + dR4)

        S_list.append(S)
        I_list.append(I)
        R_list.append(R)

    # Daily new infections
    I_curve = np.array(I_list)
    delta_I = np.maximum(np.diff(I_curve), 1e-6)

    return delta_I

# Log-posterior function for beta and gamma given observed data x_obs
def log_posterior(beta, gamma, x_obs, alpha_b=2, b_b=10,
                  alpha_g=2, b_g=10):
    if beta <= 0 or gamma <= 0:
        return -np.inf
    delta_I = simulate_sir_rk4(beta, gamma)
    log_lik = np.sum(poisson.logpmf(x_obs, delta_I))
    log_prior = ((alpha_b - 1)*np.log(beta) - b_b*beta
                 + (alpha_g - 1)*np.log(gamma) - b_g*gamma)
    return log_lik + log_prior

# Metropolis-within-Gibbs sampler
def gibbs_sampler(x_obs, n_iter=7000, burnin=2000,
                  sigma_beta=0.005, sigma_gamma=0.009):

    # Sample initial beta and gamma from prior
    beta_curr  = gamma_dist.rvs(a=.2, scale=1/10)
    gamma_curr = gamma_dist.rvs(a=.2, scale=1/10)

    samples_beta  = []
```

```
    samples_gamma = []

    gamma_acc = 0
    beta_acc = 0

    for k in range(n_iter):

        # Update beta
        beta_prop = beta_curr + np.random.normal(0, sigma_beta) #
           Metropolis proposal
        log_r = log_posterior(beta_prop, gamma_curr, x_obs) \
              - log_posterior(beta_curr, gamma_curr, x_obs)
        if np.log(np.random.uniform()) < log_r: #Accept proposal
            beta_acc += 1
            beta_curr = beta_prop

        # Update gamma
        gamma_prop = gamma_curr + np.random.normal(0, sigma_gamma) #
           Metropolis proposal
        log_r = log_posterior(beta_curr, gamma_prop, x_obs) \
              - log_posterior(beta_curr, gamma_curr, x_obs)
        if np.log(np.random.uniform()) < log_r: #Accept proposal
            gamma_acc += 1
            gamma_curr = gamma_prop

        if k >= burnin: #Burning the intial samples for CLT to hold
            samples_beta.append(beta_curr)
            samples_gamma.append(gamma_curr)

    print(f"Acceptance rate for beta: {beta_acc/n_iter:.2f}")
    print(f"Acceptance rate for gamma: {gamma_acc/n_iter:.2f}")

    return np.array(samples_beta), np.array(samples_gamma)

# Generating synthetic data
def generate_synthetic_data(beta_true=.3, gamma_true=.1, T=60):
    delta_I = simulate_sir_rk4(beta_true, gamma_true, T=T)
    x_obs = poisson.rvs(delta_I)
    return x_obs
```